

5. Assumptions & Open Questions

5.1 Assumptions made in this design

1. `order-commands` is a new topic, separate from `order-data` (1.2) — confirm with Engineering. If wrong: collapses into one bidirectional topic, a wiring change not structural.
2. `OrderRejected` carries a structured reason code distinguishing permanent from transient rejections (1.7, 1.10) — confirm with Engineering. If wrong: the reactive `PlanDeactivated` trigger (2.10) can't tell them apart, and rejection messaging (4.7) stays generic.
3. `Investment.quantity` is a monetary amount converted to shares/units at execution, not a literal share count — confirm with Product. If wrong: order semantics, rounding, and partial-fill handling all change — affects the entity model (1.9) directly.
4. A partially-executed Execution (some Filled, some Rejected) is a valid, complete, terminal outcome — nothing retries automatically. If wrong: 2.7's decision needs revisiting, and a remediation flow needs designing.

5.2 Open questions for Product

1. How does a Plan move from `Entered` to `Active` — a distinct activation step, or does creation activate it immediately? Determines whether `Entered` is a real, reachable state worth keeping at all.
2. Plan constraints: min/max investment per asset, max assets per plan, and behavior when `recurrence_day` doesn't exist in a given month. Directly affects the Dispatcher's due-plan query (1.5); needs a concrete rule.
3. Any data retention/archival requirement for Executions/Orders once a Plan is Canceled? Affects whether the read model needs an archival/purge story.
4. Does a Notification domain capable of consuming `investment-plan-data` already exist at BUX? If not, "notify the user" is a dependency on unstarted work, not something this design solves.
5. Is manually re-triggering a missed/failed execution a supported support workflow? The idempotency key (2.5) has no designed override (4.4) — needs a decision before it's a real ticket.
6. Is proactive detection of a blocked plan in scope for MVP, or explicitly deferred? The reactive-only path (2.10) is a stated trade-off (3.3), not a permanent decision.
7. Is 99.9% API availability the right target? `6-non-functional-requirements.md` 6.3 proposes it as a reasoned default, not a negotiated one.

5.3 Open questions for Engineering (mostly the Trading/Orders team)

1. Can order placement commands go on a new `order-commands` topic, or must they go through the existing `order-data` topic (5.1 #1)? Needs to be settled before build, not discovered during integration.
2. Does `OrderRejected` already carry a structured reason code (5.1 #2), or would exposing one be new scope for the Trading/Orders team? Blocks the reactive `PlanDeactivated` path and clear rejection messaging (2.10, 4.7) either way.
3. What's `order-data`'s partition count/key? Determines if the Order-Status Consumer can scale horizontally (4.6) — we don't control this topic.
4. Is optimistic concurrency expected on the Plan aggregate, or is last-write-wins acceptable for MVP? Not designed anywhere in this document; needs an explicit answer.